

Multi-Agent Deep Researcher

12-Hour Hackathon Build Plan · Team of 3 · Python + LangGraph

What you're building

A domain-scoped multi-agent research system. A user enters a research question; four specialised agents collaborate via a LangGraph state machine to retrieve sources, analyse claims, detect contradictions, and compile a cited report — all with live agent status visible in the UI.

Key differentiator	Contradiction detection: when two sources disagree, the Critic agent surfaces the specific conflicting claims side-by-side with a confidence score. No other team will have this.
--------------------	---

Agent architecture (4 agents)

Retriever	Web search via Tavily only. Returns top 5 results per query.	Tavily API · top-5 results · URL + snippet
Analyst	Summarises each source and extracts a list of discrete factual claims.	Per-source summary · claim extraction · LLM chain
Critic	Cross-references claims across all sources. Outputs conflict pairs with a 0–1 confidence score. Your core differentiator — never cut this.	Contradiction pairs · confidence 0–1 · LLM comparison
Report Builder	Assembles analyst summaries + critic findings into a structured markdown report with inline citations.	Markdown output · citations · insight bullets

Cut: Insight Generator agent — its value is absorbed into the Critic and Report Builder prompts.

Team of 3 — roles

Person A	Person B	Person C
Backend	Agents + Evals	Frontend + Demo

Person C starts the UI at hour 0 using a mocked API response — do not wait for the backend to be ready.

12-hour timeline

Person A — Backend	Person B — Agents + Evals	Person C — Frontend + Demo
--------------------	---------------------------	----------------------------

Hrs 0–1 State schema + graph skeleton Define shared state. Agree API contract with Person C. Hrs 1–4 Retriever + Analyst agents Tavily search working, per-source summarisation, claims extracted. Hrs 4–6 Wire up graph + FastAPI endpoint Connect agents in LangGraph, expose single /research POST endpoint. Hrs 6–9 Integration + retry logic End-to-end query working. Add basic error handling + timeouts. Hrs 9–12 Support + cache demo queries Buffer for integration issues. Pre-run and cache 2 demo queries.	Hrs 0–1 Write 5 test cases Research questions with known good answers. Define what correct looks like. Hrs 1–5 Critic agent Contradiction detection across source summaries. Output: conflict pairs + confidence 0–1. Hrs 5–7 Report Builder agent Markdown report with citations, contradiction section, insight bullets. Hrs 7–10 Eval harness LLM-as-judge scoring on 5 test cases. Get a number to show judges. Hrs 10–12 Prompt tuning Fix worst failures from evals. Good enough is enough — don't over-optimize.	Hrs 0–1 Streamlit scaffold + mock Query input, result panel. Use mocked JSON — don't wait for backend. Hrs 1–5 Agent status panel Live progress: which agent is running, streaming status updates. Critical for demo wow-factor. Hrs 5–8 Connect real backend Swap mock for real API. Show conflict pairs side-by-side with highlights. Hrs 8–10 Demo queries + slides Pick 2 great demo questions. Pre-cache results. Build 5-slide deck. Hrs 10–12 Rehearse demo twice Time it. Know your fallback if live query is slow. Practice the narration.
--	--	--

Hard checkpoints — do not slip these

Time	Milestone	If you miss it
Hour 1	State schema agreed, API contract signed off	A and C will build incompatible things
Hour 4	Retriever + Analyst returning real data	Critic agent has nothing to work with
Hour 7	Full pipeline: query → report working end-to-end	No time to fix integration bugs
Hour 9	UI showing real results, not mock data	Demo will fail live
Hour 10	Demo rehearsed at least once	You'll overrun or fluff the pitch

What to cut if you fall behind

Feature	Decision	Reason
Contradiction detection + UI highlight	Never cut	Your biggest differentiator
Live agent status in UI	Never cut	Critical demo wow-factor
Eval results (even 3 cases)	Never cut	Judges explicitly want this
FastAPI → call agents direct from Streamlit	Simplify if behind	Removes one integration layer
Citation linking in report	Cut if tight	Plain text refs are fine
More than 5 sources per query	Cut — hard limit 5	Quality over quantity
Multi-domain support	Cut entirely	One domain done well beats three done poorly

Recommended tech stack

Orchestration	LangGraph	State machine, conditional edges, shared state schema
Retrieval	Tavily API	Web search with clean JSON output — simplest integration
LLM	Claude claude-sonnet-4-6 or GPT-4o	For Analyst, Critic, and Report Builder agents
API layer	FastAPI	Single /research POST endpoint; stream via SSE if time allows
Frontend	Streamlit	st.status() for live agent progress; swap mock → real at hour 5
Evals	LLM-as-judge	Score coverage, accuracy, contradiction detection rate
Demo domain	AI/ML research or climate science	Contradictions are common and interesting — Critic agent has work to do

The one thing that wins it

Make the contradiction detection visible in the UI. When two sources disagree, show them side by side with the specific conflicting claims highlighted. No other team will have this — and it is the single most impressive thing a research agent can demonstrate to judges.